

Creating Databases & Tables

We've already focused on querying and reading data from existing databases and tables. Let's now shift focus to creating our own databases and tables. We will be going through some theoretical concepts for the below topics:

- Data Types
- Primary & Foreign Keys
- Constraints
- CREATE
- INSERT
- UPDATE
- DELETE, ALTER, DROP

Data Types

We are going to review some of the main data types in SQL, since when creating a table, we'll have to choose what data type each column should hold.

- **Boolean:** True or False
- **Character:** char, varchar, and text
- **Numeric:** integer and floating-point number
- **Temporal:** date, time, timestamp, and interval

Similarly, there are some uncommon SQL datatypes are :

- **UUID:** Universally Unique Identifiers
- **Array:** Stores an array of strings, numbers, etc.
- **JSON**
- Hstore key-value pair
- Special types such as network address and geometric data.

When creating databases and tables, you should carefully consider which data types should be used for the data to be stored. You can review the documentation to see limitations of data types:

[PostgreSQL: Documentation: 18: Chapter 8. Data Types](#)

For example:

- Imagine we want to store a phone number, should it be stored as numeric?
- If so, which type of numeric?

We could take a look at the above documentation for options.

- You may start to think that it makes sense to store it as a **BIGINT** data type, but we should really be thinking what is the best for the situation.
- Why bother with numerics at all ?
- We don't perform arithmetic with numbers, so it probably makes more sense as a **VARCHAR** data type instead.
- You can also search for best practice online and discover that it's usually recommended to store phone numbers as a text based data type due to a variety of issues.
 - No arithmetic performed
 - Leading zeros could cause issues, 7 and 07 are treated the same numerically, but are not the same phone number.

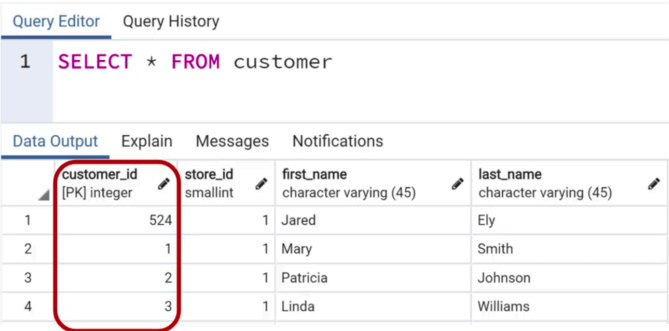
It makes more sense to use a text data type like varchar rather than numerics.

Primary & Foreign Keys

Primary Keys

- A primary key is a column or a group of columns used to identify a row uniquely in a table. For example, in our dvdrental database we saw customers had a unique, non-null `customer_id` column as their primary key.
- A primary key is also important since they allow us to easily discern what columns should be used for joining tables together.

Example of primary key :



The screenshot shows a database query editor with a query window and a results window. The query is `SELECT * FROM customer`. The results window displays a table with columns: `customer_id` (integer, PK), `store_id` (smallint), `first_name` (character varying (45)), and `last_name` (character varying (45)). The `customer_id` column is highlighted with a red box, indicating it is the primary key.

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)
1	524	1	Jared	Ely
2	1	1	Mary	Smith
3	2	1	Patricia	Johnson
4	3	1	Linda	Williams

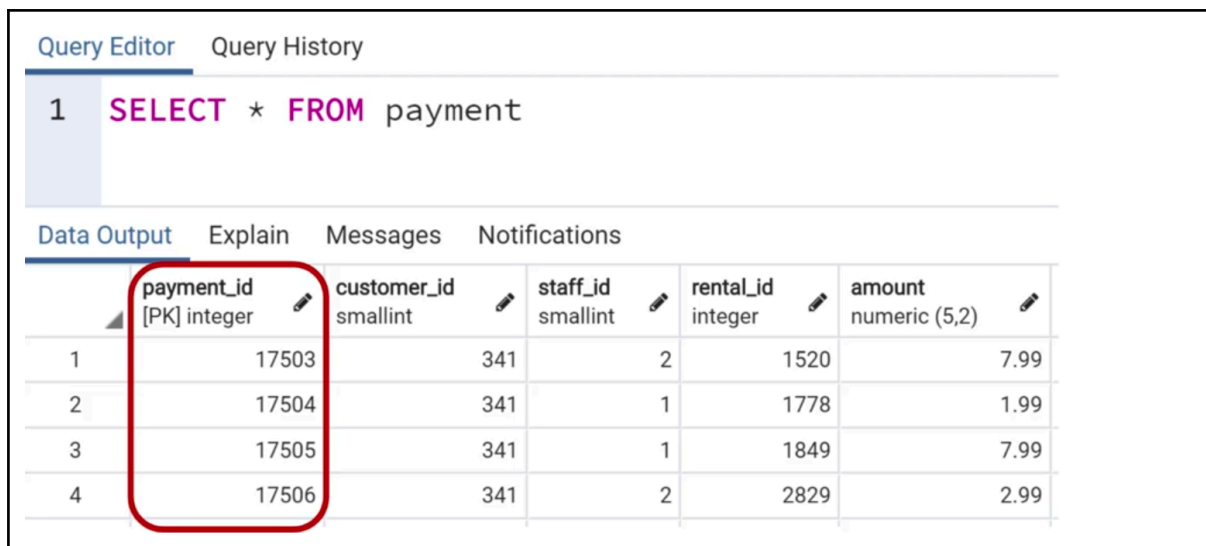
The customer_id is the primary key for the above customer table, the table will tell you if a column is the primary key or not by specifying it with **PK**. Also notice that the primary key is a unique integer and is also not null (every row for customer_id must have an entry). So this is a customer_id integer that uniquely identifies each customer.

You wouldn't want to use something like the first_name as a primary key because people can share the same first_name. However they cannot share this unique integer ID.

Foreign Keys

- A foreign key is a field or group of fields in a table that uniquely identifies a row in another table.
- A foreign key is defined in a table that references the primary key of the other table.
- The table that contains the foreign key is called a referencing table or child table.
- The table to which the foreign key references is called a referenced table or parent table.
- A table can have multiple foreign keys depending on its relationships with other tables.

For Example:



The screenshot shows a database interface with a query editor and a results table. The query editor displays the following SQL statement:

```
1 SELECT * FROM payment
```

The results table has the following columns and data:

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)
1	17503	341	2	1520	7.99
2	17504	341	1	1778	1.99
3	17505	341	1	1849	7.99
4	17506	341	2	2829	2.99

payment_id is the primary key for the payment table. You can also see other columns (customer_id, staff_id & rental_id) which are actually the foreign key references.

Query Editor

Query History

1

SELECT * FROM payment

Data Output

Explain

Messages

Notifications

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)
1	17503	341	2	1520	7.99
2	17504	341	1	1778	1.99
3	17505	341	1	1849	7.99
4	17506	341	2	2829	2.99

The reason these columns are foreign keys is because they are references to a primary key in another table.

- customer_id is the primary key from the customer table.
- staff_id is the primary key from the staff table.
- rental_id is the primary key from the rental table.

Note that the pgAdmin won't alert you directly to FK in a simple query call. They only do that for primary keys.

You may begin to realize primary key and foreign key typically make good column choices for joining together two or more tables.

When creating tables and defining columns, we can use constraints to define columns as beginning a primary key , or attaching a foreign key relationship to another table.

Constraints

Constraints are the rules enforced on data columns on tables. These are used to prevent invalid data from being entered into the database. This ensures the accuracy and reliability of the data in the database.

Constraints can be divided into two main categories:

- Column Constraints:
 - Constraints the data in a column to adhere to certain conditions.
- Table Constraints:
 - Applied to the entire table rather than to an individual column.

The most common constraints used:

- **NOT NULL** Constraint
 - Ensures that a column cannot have NULL value.
- **UNIQUE** Constraint
 - Ensures that all values in a column are different.
- **PRIMARY Key**
 - Uniquely identifies each row/record in a database table.
- **FOREIGN Key**
 - Constrains data based on columns in other tables.
- **CHECK** Constraint
 - Ensures that all values in a column satisfy certain conditions.
- **EXCLUSION** Constraint
 - Ensures that if any two rows are compared on the specified column or expression using the specified operator, not all of these comparisons will return TRUE.
- **CHECK (condition)**
 - To check a condition when inserting or updating data.
- **REFERENCES**
 - To constrain the value stored in the column that must exist in a column in another table.
- **UNIQUE (column_list)**
 - Forces the values stored in the columns listed inside the parentheses to be unique.
- **PRIMARY KEY (column_list)**
 - Allows you to define the primary key that consists of multiple columns.

CREATE

In SQL you can create a table using the **CREATE** keyword and column syntax.

Full General Syntax:

```
CREATE TABLE table_name (  
    column_name TYPE column_constraint,  
    column_name TYPE column_constraint,  
    table_constraint table_constraint  
) INHERITS existing_table_name;
```

Common simple syntax

```
CREATE TABLE players (  
    column_name TYPE column_constraint,  
    column_name TYPE column_constraint  
);
```

Example syntax:

```
CREATE TABLE players(  
    player_id SERIAL column_constraint,  
    column_name TYPE column_constraint,  
);
```

- **SERIAL**

- In PostgreSQL, a sequence is a special kind of database object that generates a sequence of integers.
- A sequence is often used as the primary key column table.
- It creates a sequence object and sets the next value generated by the sequence as the default values for the column.
- This is perfect for a primary key, because it logs unique integer entries for you automatically upon insertion.
- If a row is later removed, the column with the SERIAL data type will not adjust, marking the fact that a row was removed from the sequence, for example:

- 1,2,3,5,6,7
 - You know row 4 was removed at some point

Name	Storage Size	Description	Range
smallint	2 bytes	small-range integer	-32768 to +32767
integer	4 bytes	typical choice for integer	-2147483648 to +2147483647
bigint	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
decimal	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
numeric	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
real	4 bytes	variable-precision, inexact	6 decimal digits precision
double precision	8 bytes	variable-precision, inexact	15 decimal digits precision
smallserial	2 bytes	small autoincrementing integer	1 to 32767
serial	4 bytes	autoincrementing integer	1 to 2147483647
bigserial	8 bytes	large autoincrementing integer	1 to 9223372036854775807

We are using the **SERIAL** type for player_id as it's the primary key for this table and also the constraint as **PRIMARY KEY**.

```
CREATE TABLE players(
  player_id SERIAL PRIMARY KEY,
  column_name TYPE column_constraint,
);
```

Add the last column as age:

- We will use the type as **SMALLINT** as we won't need such a huge integer value range for ages.
- Column constraint can be whatever you want, lets constraint the age column using **NOT NULL**, meaning that if you're inserting a player into this table you cannot leave the age blank.

```
CREATE TABLE players(
  player_id SERIAL PRIMARY KEY,
  age SMALLINT NOT NULL
);
```

INSERT

INSERT allows you to add rows to a table.

General syntax:

```
INSERT INTO table (column1, column2, ....)
VALUES
    (value1, value2, .....),
    (value1, value2, .....), .....
```

Syntax for inserting values from another table:

```
INSERT INTO table(column1, column2, ....)
SELECT column1, column2, .....
FROM another_table
WHERE condition;
```

- Keep in mind, the inserted row values must match up for the table, including constraints.
- **SERIAL** columns do not need to be provided a value.

UPDATE

The **UPDATE** keyword allows for the changing of values of the columns in a table.

General Syntax:

```
UPDATE table
SET column1 = value1,
      column2 = value2, ....
WHERE
      condition;
```

Example:

```
UPDATE account
SET last_login = CURRENT_TIMESTAMP
WHERE last_login IS NULL;
```

You can also reset everything without the WHERE condition:

```
UPDATE account
SET last_login = CURRENT_TIMESTAMP;
```

You can also update the table using another's table values (UPDATE join):

```
UPDATE TableA
SET original_col = TableB.new_col
FROM tableB
WHERE tableA.id = TableB.id;
```

If you want to return some rows after updating, you can use the below syntax:

```
UPDATE account
SET last_login = created_on
RETURNING account_id, last_login;
```

DELETE

- We can use the DELETE clause to remove rows from a table.

Syntax Example:

```
DELETE FROM table  
WHERE row_id = 1;
```

- We can also delete rows based on their presence in other tables:

Syntax Example:

```
DELETE FROM tableA  
USING tableB  
WHERE tableA.id = tableB.id;
```

- We can also delete all rows from a table

Syntax example:

```
DELETE FROM table;
```

Similarly to the **UPDATE** command, you can also add in a **RETURNING** call to return rows that were removed.

ALTER

The **ALTER** clause allows for changes to an existing table structure, such as :

- Adding, dropping, or renaming columns
- Changing a column's data type
- Set **DEFAULT** values for a column
- Add **CHECK** constraints
- Rename table

General Syntax:

```
ALTER TABLE table_name action;
```

Adding new columns:

```
ALTER TABLE table_name  
ADD COLUMN new_col TYPE;
```

Dropping columns:

```
ALTER TABLE table_name  
DROP COLUMN col_name;
```

Alter constraints:

- Set a default value

```
ALTER TABLE table_name  
ALTER COLUMN col_name  
SET DEFAULT value;
```

- Drop the default value

```
ALTER TABLE table_name  
ALTER COLUMN col_name  
DROP DEFAULT;
```

- Add not null constraint

```
ALTER TABLE table_name  
ALTER COLUMN col_name  
SET NOT NULL;
```

- Remove not null constraint

```
ALTER TABLE table_name  
ALTER COLUMN col_name  
DROP NOT NULL;
```

- Add a general constraint

```
ALTER TABLE table_name  
ALTER COLUMN col_name  
ADD CONSTRAINT constraint_name;
```

DROP

DROP allows for the complete removal of a column in a table. In PostgreSQL this will also automatically remove all of its indexes and constraints involving the column. However, it will not remove columns used in views, triggers, or stored procedures without the additional **CASCADE** clause.

General Syntax:

```
ALTER TABLE table_name  
DROP COLUMN col_name CASCADE;
```

Check for existence to avoid error:

```
ALTER TABLE table_name  
DROP COLUMN IF EXISTS col_name;
```

Drop multiple columns:

```
ALTER TABLE table_name  
DROP COLUMN col_one  
DROP COLUMN col_two;
```

CHECK

The **CHECK** constraint allows us to create more customized constraints that adhere to a certain condition. Such as making sure all inserted integer values fall between a certain threshold.

General Syntax:

```
CREATE TABLE example(  
    ex_id SERIAL PRIMARY KEY,  
    age SMALLINT CHECK (age > 21),  
    parent_age SMALLINT CHECK (parent_age > age)  
)
```